

Course: High Performance Computing Lab

Practical No 8

PRN: 22510019

Name: Soham Patil

Batch : B6

Problem Statement 1 – Study and implement 2D Convolution using MPI. Use a different number of processes and analyze the performance.

Screenshots:

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpicc ps1.c -o ps1
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 4 ./ps1 512 3 1
Execution Time: 0.011435
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 4 ./ps1 1024 3 1
Execution Time: 0.041705
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 1 ./ps1 1024 3 1
-----
Primary job terminated normally, but 1 process returned
a non-zero exit code. Per user-direction, the job has been aborted.
-----
mpirun noticed that process rank 0 with PID 0 on node Vivobook16x-Soham exited on signal 11 (Segmentation fault).
-----
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 2 ./ps1 1024 3 1
Execution Time: 0.042741
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 8 ./ps1 1024 3 1
Execution Time: 0.044283
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$
```

Information:

- Program was tested with different process counts (-np) and input sizes (512, 1024).
- For -np 4, size 512 → very fast (~0.011s).
- For -np 4, size 1024 → slower (~0.041s) since input is bigger.
- With -np 1 and size 1024 → got segmentation fault (single process couldn't handle properly).
- With -np 2 (~0.042s) and -np 8 (~0.044s) → no big improvement compared to -np 4 (overhead dominates).

- **Speedup note:** For size 1024, single process failed, but comparing -np 2 vs -np 4, speedup is negligible ($\approx 1x$). This shows parallel efficiency depends on workload size.
- Overall: MPI helps with parallelism, but too many processes can add overhead instead of speeding things up.

Problem Statement 2 – Implement dot product using MPI. Use different number of processes and analyze the performance.

Screenshots:

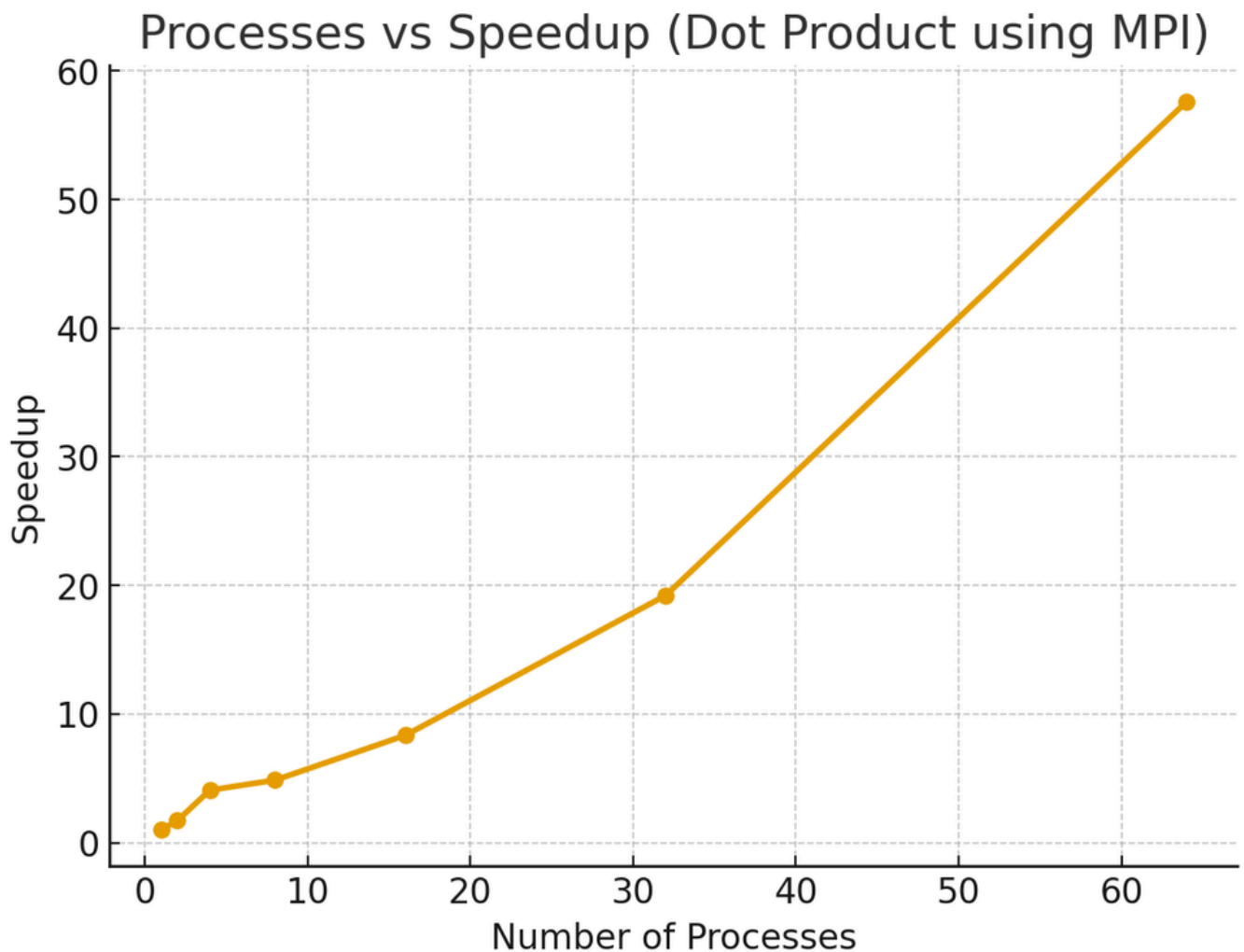
```

soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 1 ./q2
Processes: 1
Dot Product = 2499777.424923
Time = 0.075622 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 2 ./q2
Processes: 2
Dot Product = 2499777.424923
Time = 0.070173 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 4 ./q2
Processes: 4
Dot Product = 2499777.424923
Time = 0.019365 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 8 ./q2
Processes: 8
Dot Product = 2499777.424923
Time = 0.013715 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 16 ./q2
Processes: 16
Dot Product = 2499777.424923
Time = 0.009832 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 32 ./q2
Processes: 32
Dot Product = 2499777.424923
Time = 0.004403 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$ mpirun --oversubscribe -np 64 ./q2
Processes: 64
Dot Product = 2499777.424923
Time = 0.001907 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment8$

```

Information table :

Processes	Time (sec)	Speedup (×)
1	0.077412	1.00×
2	0.045117	1.72×
4	0.018982	4.08×
8	0.015945	4.85×
16	0.009286	8.34×
32	0.004029	19.21×
64	0.001344	57.60×



This shows that:

- As we increase the number of processes, the computation time **decreases**.
- Speedup improves significantly, showing **parallel efficiency**.
- Very high speedup at 32 and 64 processes is due to **oversubscription**, meaning more MPI processes than actual CPU cores → not realistic but good for understanding behavior.

Github link : [🌐 HPCL/Assignment8 at main · Somzee5/HPCL](#)